this is heading 1

this is heading 2

this is heading 3

this is first `true` 1

- bullet level 1

  - bullet level 2
    - bullet level 3
    - bullet level 3
    - bullet level 3

this is first `true` 1

- bullet level 1

  - bullet level 2
    - bullet level 3
    - bullet level 3
    - bullet level 3

this is first `true` 1

- bullet level 1

- **bullet level 2**
  - bullet level 3
  - bullet level 3
  - bullet level 3

this is first `true` 1

- **bullet level 1**

- **bullet level 2**
  - bullet level 3
  - bullet level 3
  - bullet level 3

1. this is number list **this is first `true` 1**
   - **this must be level 2 list item**

     - **bullet level 3**

     - **bullet level 3**

     - **bullet level 3**

1. this is number list **this is first `true` 1**
   - **this must be level 2 list item**

- bullet level 3
- bullet level 3
- bullet level 3

1. this is number list **this is first `true` 1**
   - **this must be level 2 list item**
     - bullet level 3
     - bullet level 3
     - bullet level 3

하흥주: Frontend Developer
트렌드보다 본질, 복잡함보다 명료함
문제를 정확히 저의하고, 단순한 해결책을 만듭니다
010-6332-9205
`nusilite@gmail.com`
https://github.com/thepott

Arbor The Tree 진도 관리 시스템으로

해결하려고 한 문제

1. 입시학원의 문제
2. 구글 시트 + 파이썬 시스템의 문제
3. 고객의 요구사항

해결한 과제

1. 쉬운 사용법 **this is first `true` 1**
   - **칸반 진도표**

     this is first `true` 1
   - **그리드 인풋**
2. 병목 현상 해결 **this is first `true` 1**
   - **진도표와의 동기화**
3. 성능 최적화 **this is first `true` 1**
   - **PDF 성능**

     this is first `true` 1
   - **서버 응답시간 단축**

Arbor The Tree 진도 관리 시스템으로

해결하려고 한 문제

1. 입시 학원의 문제

this is first `true` 1

- 강사 기억에 의존한 진도 추적

this is first `true` 1

- 비정량적 진도

this is first `true` 1

- 오답 복습 관리 불가

2. 구글 시트 + 파이썬 시스템의 문제

this is first `true` 1

- 이를 해결하려 파이썬과 구글 시트로 진도 관리 시스템 제작

this is first `true` 1

- CLI로 제작되어 사용자 친화성 부족

this is first `true` 1

- 병렬적 자료 생성, 직렬적 동기화로 병목 현상 발생

3. 고객의 요구사항

this is first `true` 1

- 빠른 반응 속도 요구

this is first `true` 1

- 쉬운 조작법 요구

Arbor The Tree 진도 관리 시스템으로 해결하려고 한 문제

1. 입시 학원의 문제

this is first `true` 1

- 강사 기억에 의존한 진도 추적
  - 여러 학년을 한 반에서 가르치는 무학년 수업에서 두드러짐
  - 학생의 개별 성취도를 파악하기가 어려움

this is first `true` 1

- 비정량적 진도
  - 한 수업에서 할 양이 정해져있지 않음
  - 앞으로 몇 번의 수업이 더 필요한지 예상 불가

this is first `true` 1

- 오답 복습 관리 불가

- 학생이 오답 복습을 했는지 확인할 방법이 없음
- 복습 대신 새 문제집만 풀리고 학생은 틀리는 문제는 계속 틀림

Arbor The Tree 진도 관리 시스템으로 해결하려고 한 문제

2. 구글 시트 + 파이썬 시스템의 문제

this is first `true` 1

- 이를 해결하려 파이썬과 구글 시트로 진도 관리 시스템 제작

this is first `true` 1

- CLI로 제작되어 사용자 친화성 부족
  - 다른 강사들이 사용하지 못함

this is first `true` 1

- 병렬적 자료 생성, 직렬적 동기화로 병목 현상 발생
  - 학생들이 개별로 오답체크, 진도표에 완료 표시 -> 강사가 동기화
  - 동기화에 한 학생당 5분, 한 수업당 50 100분 소요

Arbor The Tree 진도 관리 시스템으로 해결하려고 한 문제

3. 고객의 요구사항

this is first `true` 1

- **빠른 반응 속도 요구**

this is first `true` 1

- **쉬운 조작법 요구**

  - **아르바이트생의 잦은 교체에도 사용법 인수인계가 원활히 되어야**
    - 고객(원장)은 강의에만 집중 희망
    - 스스템 운용은 아르바이트생에게 위임 계획

Arbor The Tree 진도 관리 시스템으로

해결한 과제

1. 쉬운 사용법

a. 칸반 진도표

b. 그리드 인풋

2. 병목 현상 해결

a. 오답 체크 결과 자동 반영

3. 성능 최적화

a. 서버 사이드 PDF 생성

b. 서버 응답 시간 단축

Arbor The Tree로 해결한 과제

1. 쉬운 사용법

a. 칸반 진도표

this is first `true` 1

- 기존 진도표의 문제와 그 해결

this is first `true` 1

- Troubleshooting: 드롭다운이 스크롤바에 잘림

b. 그리드 인풋

this is first `true` 1

- 기존 문제집 등록 과정의 문제

this is first `true` 1

- 후보 기술 스택

this is first `true` 1

- 문제의 본질 파악

this is first `true` 1

- Troubleshooting: 유효성 검사 성능 개선

this is first `true` 1

- Troubleshooting: 키보드 내비게이션

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

a. 칸반 진도표

기존 진도표의 문제와 그 해결

this is first `true` 1

- 기존 진도표 사용 흐름
  1. 구글 시트에 문제 묶음을 복사 붙여넣기
  2. 묶음 이름 옆에 진행 상황을 키워드로 기록
  3. 이를 출력하여 학생 책상 위에 배치

this is first `true` 1

- 기존 진도표의 문제점
  1. 전체 진도표를 한 페이지에 넣다보니 글씨가 너무 작아짐
  2. 당일 수업과 무관한 것이 페이지의 대부분
  3. 조건부 포매팅 규칙을 지켜야만 사용 가능

this is first `true` 1

- 해결책

- **열(column)별 분리 스크롤**
  - 문제집별 스크롤 칸반 보기

- **드롭다운에서 묶음 상태 설정 (숙제, 오늘, 해제)**

- **당일 수업에 필요한 부분만 요약 보기**

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

a. 칸반 진도표

Troubleshooting: 드롭다운이 스크롤바에 잘림

this is first `true` 1

- 현상 관찰
  - 열 컴포넌트의 자식으로 들어간 드롭다운이 스크롤바에 잘림

this is first `true` 1

- 해결책
  - `createPortal`을 이용해 컴포넌트를 `document.body`로 빼냄
  - `position: fixed` 하에서의 위치는 Floating UI 라이브러리로 계산

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

b. 그리드 인풋

기존 문제집 등록 과정과 그 문제

this is first `true` 1

- 기존 문제집 등록 흐름
  - 구글 시트에 문제의 id, 쪽, 문제 번호 등의 정보를 기록
  - 반복적으로 입력해야 하는 부분에서는 자동 채우기 혹은 formula 사용

this is first `true` 1

- 개선해야 했던 것
  - 작성 중에는 놓친 부분이 있는지 파악하기 어려움
    - formula로 다 채우기 이전에는 변경 지점이 /로만 보임
  - 유효성 검사 부재
    - 아르바이트생에게 맡기기에 걱정됨

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

b. 그리드 인풋

후보 기술 스택

this is first `true` 1

- 기존의 작업 환경인 구글 시트와 유사한 UI 제작하려 함

  - AG Grid (부분 유료, 1년 999 달러)
    - 다중 선택, formula는 유료 기능

  - Jspreadsheet CE (무료)
    - 바닐라 JavaScript 라이브러리
    - 선언적으로 컴포넌트 생성 삭제가 불가능

  - TanStack Table (무료)
    - Headless UI Library
    - 모든 셀, 행, 열의 데이터에 언제든 접근 가능

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

b. 그리드 인풋

문제의 본질

this is first `true` 1

- 필요한 기능들은 스프레드 시트에 종속되지 않는다
  - 1씩 증가 자동 채우기: 마우스 드래그를 할 필요가 없다
  - 빈 곳 채우기: formula가 아니어도 채워지면 된다
  - 변환값 미리 보기: spreadsheet에 원래 없는 기능이다
  - 유효성 검사: spreadsheet에 원래 없는 기능이다

this is first `true` 1

- 결론: TanStack Table로 직접 구현
  - 테이블 형식이 필요한 것이지 스프레드 시트가 필요한 것이 아니다

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

b. 그리드 인풋

Troubleshooting: 유효성 검사 성능 개선

this is first `true` 1

- 문제 발생: 느린 유효성 검사
  - 유효성 검사를 하는 셀을 최소화하려다보니 예외 규칙과 더불어 버그가 많아짐
  - 검사 안정성 높이기 위해 모든 셀 검사
  - 2000행 x 7열의 돔 노드가 업데이트

this is first `true` 1

- 해결: 가상 스크롤
  - TanStack Virtual로 돔 노드 최소화

- 업데이트 시간을 20.28ms에서 2.28ms로 89% 단축 (`performance.time`으로 측정)

Arbor The Tree로 해결한 과제 1. 쉬운 사용법

b. 그리드 인풋

Troubleshooting: 키보드 내비게이션

this is first `true` 1

- 문제 발생: 구글 시트보다 조악한 키도드 내비게이션
  - 모든 셀은 `<input />`일 뿐이라 키보드 이동은 tab만 지원
  - 방향키로 이동 불가
  - 엔터로 아래 셀 이동 불가

this is first `true` 1

- 해결: custom data attribute
  1. `data-coordinate={...}` 부여

2. 방향키, 엔터 입력 시 다음 coordinate 계산
3. querySelector로 목적지 돔 노드 선택
4. focus

Arbor The Tree로 해결한 과제

2. 병목 현상 해결

a. 오답 체크 결과 자동 반영

this is first `true` 1

- 병목 현상 발생 지점과 그 해결

this is first `true` 1

- UI 벤치마크: 호텔 예약의 다중 선택 UI

this is first `true` 1

- Troubleshooting: 다층 구조 가상 스크롤

Arbor The Tree로 해결한 과제 2. 병목 현상 해결

a. 칸반 진도표

병목 현상 발생 지점과 그 해결

this is first `true` 1

- 수업 종료 시의 기록물에서 병목 현상 발생
  - 학생이 개별적으로 오답 체크
    - 강사가 모아서 구글 시트에 기록
  - 학생이 개별적으로 인쇄된 진도표에 완료 표시
    - 강사가 모아서 구글 시트에 동기화

this is first `true` 1

- 해결 방안: 오답 체크와 DB, 진도표 동기화
  1. 웹앱에 오답 체크 -> DB에 저장
  2. DB에 오답 체크 저장 -> 묶음 완료 여부 업데이트
  3. 진도표 캐시 무효화 -> 묶음 완료 여부 업데이트 된 진도표 fetch

Arbor The Tree로 해결한 과제 2. 병목 현상 해결

a. 오답 체크 결과 자동 반영

UI 벤치마크: 호텔 예약의 다중 선택 UI

this is first `true` 1

- 요구사항: 모바일에서의 한 페이지에서 한 수업 분량 오답 체크 가능해야 가능해야
  - 한 수업 당 약 40문제 오답 체크
  - 기존 방식인 한 줄에 한 문제 체크로는 스크롤을 너무 많이 해야 함

this is first `true` 1

- 아이디어: 호텔 예약의 다중 선택 UI
  - 그리드 체크박스를 만들고 시작과 끝만 선택하면 그 사이도 자동 선택되게
  - 이후 단일 선택으로 다른 상태 기록할 수 있게

Arbor The Tree로 해결한 과제 2. 병목 현상 해결

a. 오답 체크 결과 자동 반영

Troubleshooting: 다층 구조 가상 스크롤

this is first `true` 1

- 배경: 다층 구조의 데이터 스키마
  - book -> topic -> step -> question

this is first `true` 1

- 문제 발생: 가상 스크롤 적용시, 가장 바깥의 부모가 한 덩어리로 가상화됨
  - 거대한 부모가 통째로 mount되거나 unmount됨. 눈에 보이는 한 줄 씩 가상화가 불가능

this is first `true` 1

- 문제의 원인: 눈에 보이는 대로 줄별로 끊어지지 않은 다층 구조

this is first `true` 1

- 해결: 다층 구조를 눈에 보이는 줄 별로 평탄화, 이후 가상 스크롤

Arbor The Tree로 해결한 과제

3. 성능 최적화

a. 서버 사이드 PDF 생성

this is first `true` 1

- PDF 생성 방법과 그 문제

this is first `true` 1

- 문제의 본질 파악과 그 해결

this is first `true` 1

- Troubleshooting: `npm install`로 설치 안 되는 것들 존재

b. 서버 응답 시간 단축

this is first `true` 1

- 원인 진단: region, cold start

this is first `true` 1

- 해결과 그 성과

Arbor The Tree로 해결한 과제 3. 성능 최적화

a. 서버 사이드 PDF 생성

PDF 생성 방법과 그 문제

this is first `true` 1

- 배경: 오답과제를 pdf로 생성 후 출력 후 숙제로 부여
  - 학생들의 오답의 쪽, 문제 번호가 적혀있는 빈 학습지 제작해야
  - 여러 학생 것을 한 번에 제작한다면 100쪽 이상도 가능

this is first `true` 1

- 후보 기술 스택
  - React PDF(client side)
    - 브라우저 메인 스레드 점유
    - 30쪽 이상의 pdf 생성 시에는 Web Worker 사용 권장
  - Puppeteer(server side)
    - 브라우저 가상화 도구에 pdf 생성 기능이 딸려있을 뿐
  - Typst(server side)

- LaTex보다 빠르고 문법이 쉬운 문서 조판 언어

Arbor The Tree로 해결한 과제 3. 성능 최적화
a. 서버 사이드 PDF 생성
문제의 본질 파악과 그 해결

this is first `true` 1

- 문제의 본질: PDF는 문서다
  - 브라우저와도, html과도 무관하다

this is first `true` 1

- 해결
  1. 오답 과제 pdf 생성 request -> 오답 과제 내의 문제 쿼리
  2. Typst template 생성 -> PDF로 컴파일
  3. respond with PDF

this is first `true` 1

- 성과(ReactPDF 대비)
  - 8쪽 생성 시간 4.7% 단축
  - 80쪽 생성 시간 30.7% 단축
  - 800쪽 생성 시간 61.3% 단축

- **PDF 생성이 브라우저 메인 스레드를 점유하지 않아 사용자 조작에 반응**

Arbor The Tree로 해결한 과제 3. 성능 최적화
b. 서버 응답 시간 단축
원인 진단: region, cold start

this is first `true` 1

- 시스템 구성
  - API 서버: Railway
  - DB 서버: Neon(PostgreSQL)

this is first `true` 1

- 문제 발생: 문제집 등록 시, 7초 가량이 걸림
  - 연속적으로 요청 보낼 시, 응답 시간이 4 5초로 줄어듦
  - 최초의 긴 응답 시간은 cold start 의심됨
  - 두 번째 이후에도 긴 응답 시간은 region 자체가 먼 것이 의심됨

this is first `true` 1

- 진단 및 해결 시도

- **Railway region 변경: 달라지지 않음**

- **Neon은 설정 불가**
  - 무료 플랜은 cold start 강제
  - region은 N. Virginia 하나

Arbor The Tree로 해결한 과제 3. 성능 최적화
b. 서버 응답 시간 단축
해결: PostgreSQL DB on Railway

this is first `true` 1

- **DB 서버를 PostgreSQL로 Railway에 직접 배포**
  - **Railway는 상시 가동 인스턴스로 cold start 없음**

this is first `true` 1

- **API 서버와 DB 서버의 network latency 최소화**
  - **모두 한국과 제일 가까운 Singpore region으로 설정**
  - **같은 private network로 설정하여 roundtrip latency 단축**

this is first `true` 1

- **성과**
  - **문제집 생성 시간 315 ms로 단축 (22배 향상)**

링크

this is first `true` 1

- https://arbor-the-tree-production.up.railway.app/

this is first `true` 1

- https://github.com/ThePott/arbor-the-tree

this is first `true` 1

- 테스트 계정 로그인 페이지

this is first `true` 1

- pdf 생성 성능 비교 페이지

테스트 계정

계정	ID	Password
원장	`test12@test.test`	`test1234!@#$`
실장	`test13@test.test`	`test1234!@#$`
학생	`test14@test.test`	`test1234!@#$`